

EduID Documentation

Unified Student Digital Identity & Academic Credential Verification System

Repository: Shiva-Prasad-ai/EduID Version: 1.0.0 Enterprise Security: Zero-Secrets Client Architecture

1. Executive Summary & Core Vision

EduID is an enterprise-grade digital identity platform created to modernize student credential management, academic history tracking, extracurricular verification, and AI-assisted career advisory into a secure, tamper-proof, verified digital registry.

Key Architectural Innovation: The platform enforces a *Zero-Secrets Frontend Architecture*. MongoDB connection strings, OpenRouter LLM credentials, and user password hashes reside exclusively on the server. Browser DevTools Inspect reveals no sensitive keys.

2. The EduID Specification Standard

Every registered student receives a deterministic, collision-free identity string formatted systematically:

EU
NAMESPACE (EDUID)

KA
STATE CODE (KARNATAKA)

2026
ADMISSION YEAR

001
3-DIGIT NON-OCCURRING
SERIAL

SEGMENT	TYPE / LENGTH	EXAMPLE	SPECIFICATION DESCRIPTION
Prefix	2 Characters	EU	Educational Universal namespace identifier.
State Code	2 Characters	KA	Official 2-letter uppercase ISO/postal state code (e.g. KA, MH, DL, TN).
Year	4 Digits	2026	Four-digit enrollment cohort year.
Unique Serial	3 Digits	001	Non-occurring 3-digit number (001-999) verified collision-free in MongoDB.

Storage & Normalization: Database primary key uses the compact format EUKA2026001. The presentation format uses EU-KA-2026-001. The backend normalizer cleans all punctuation and case on login, resolving

euka2026001, EU-KA-2026-001, and EUKA2026001 identically.

3. Core Code Implementations Explained

3.1. Collision-Free EduID Generator Engine

Located in `backend/utils/eduIdGenerator.js`, this function ensures zero sequence collisions by querying existing user records and selecting only from remaining unallocated slots.

BACKEND/UTILS/EDUIDGENERATOR.JS

JAVASCRIPT (NODE.JS)

```
export async function generateUniqueEduId(state = 'KA', year = 2026, randomize = true) { const
cleanState = state.trim().toUpperCase().slice(0, 2); const cleanYear = Number(year) || 2026; // 1.
Retrieve all issued sequence numbers for state + year const existingUsers = await User.find( { state:
cleanState, year: cleanYear }, { sequenceNumber: 1 } ).lean(); const usedNumbers = new
Set(existingUsers.map(u => u.sequenceNumber)); const availableNumbers = []; for (let i = 1; i <= 999;
i++) { if (!usedNumbers.has(i)) availableNumbers.push(i); } // 2. Deterministic or randomized
unallocated slot selection const selected = randomize ? availableNumbers[Math.floor(Math.random() *
availableNumbers.length)] : availableNumbers[0]; const padded = String(selected).padStart(3, '0');
return { eduId: `EU${cleanState}${cleanYear}${padded}`, formattedEduId:
`EU-${cleanState}-${cleanYear}-${padded}`, sequenceNumber: selected }; }
```

3.2. Secure Authentication & Credential Stripping

Located in `backend/routes/auth.js`, this endpoint verifies credentials via `bcrypt`. Passwords and internal database keys are completely removed before the response is dispatched.

BACKEND/ROUTES/AUTH.JS

EXPRESS.JS ROUTE

```
router.post('/login', async (req, res) => { const { eduId, password, role } = req.body; const
compactInput = eduId.trim().replace(/[^a-zA-Z0-9]/g, '').toUpperCase(); // Flexible lookup matching
either compact or hyphenated format const user = await User.findOne({ $or: [{ eduId: compactInput }, {
formattedEduId: eduId.trim() }] }); if (!user || !(await bcrypt.compare(password, user.password))) {
return res.status(401).json({ success: false, message: 'Invalid credentials' }); } // Respond with
sanitized data (Password and DB keys NEVER sent to client) res.status(200).json({ success: true, user: {
eduId: user.eduId, formattedEduId: user.formattedEduId, name: user.name, role: user.role, department:
user.department, institution: user.institution, status: user.status } }); });
```

3.3. Server-Side AI Chat Proxy

Located in `backend/routes/ai.js`, this endpoint acts as a secure reverse proxy for Jade AI, ensuring OpenRouter API tokens remain strictly on the server.

BACKEND/ROUTES/AI.JSEXPRESS.JS PROXY ROUTE

```
router.post('/chat', async (req, res) => { const { messages } = req.body; const attempts = [ { key: process.env.GEMMA_API_KEY, model: 'google/gemma-3-27b-it' }, { key: process.env.NEMOTRON_API_KEY, model: 'openrouter/auto' } ].filter(a => Boolean(a.key)); for (const attempt of attempts) { const response = await fetch('https://openrouter.ai/api/v1/chat/completions', { method: 'POST', headers: { 'Authorization': `Bearer ${attempt.key}`, 'Content-Type': 'application/json' }, body: JSON.stringify({ model: attempt.model, messages, max_tokens: 400 }) }); if (response.ok) { const data = await response.json(); return res.status(200).json({ success: true, content: data.choices[0].message.content }); } } res.status(502).json({ error: 'AI providers currently unavailable' }); });
```

4. Security & Protection Matrix

THREAT / ATTACK VECTOR	MITIGATION MECHANISM IMPLEMENTED	TARGET FILE
Client API Key Extraction	Removed keys from frontend; queries proxied via server endpoint.	src/components/AICareerAdvisor.jsx
Database URI Git Leak	Added strict ignore patterns for .env and backend/.env in .gitignore.	.gitignore
Plaintext Password Vulnerability	All passwords salted and hashed using bcryptjs (10 rounds).	backend/routes/auth.js
Windows DNS SRV Connection Failure	Configured Google DNS fallback (8.8.8.8) preventing SRV lookup failure.	backend/server.js
GitHub Pages Asset 404s	Configured relative base path (base: './') for Vite build output.	vite.config.js

5. Deployment & Execution Reference

Demo Credentials for Evaluation:

- EduID:** EUKA2026001 (or EU-KA-2026-001)
- Password:** sample user

Local Development: Run `start.bat` or run frontend (port 5173) and backend (port 5001) in separate shells.

GitHub Pages Deployment: Run `npm run deploy` to compile and publish to the gh-pages branch.